

Lección 2. Otras arquitecturas de redes neuronales

Introducción más allá de las MLP

Las redes MLP, o perceptrones multicapa, son una de las arquitecturas más conocidas dentro del aprendizaje profundo. Su estructura es sencilla pero muy potente para resolver tareas de clasificación y regresión, especialmente cuando se trabaja con datos tabulares o vectores de características. Gracias a su capacidad para combinar capas y funciones de activación, pueden aprender patrones no lineales y extraer representaciones útiles de los datos.

Una red MLP es un tipo de red neuronal feedforward, lo que significa que la información fluye siempre hacia adelante, desde la capa de entrada hacia la capa de salida, pasando por las capas ocultas. No existen ciclos ni retroalimentación entre capas. Sin embargo, durante el entrenamiento, se utiliza un proceso llamado retropropagación del error (backpropagation), que permite ajustar los pesos recorriendo la red en sentido inverso. Esta combinación de arquitectura directa y aprendizaje hacia atrás permite a las MLP aprender relaciones complejas entre los datos.

A pesar de su utilidad, las MLP presentan limitaciones importantes. Cuando se enfrentan a datos con estructura espacial (como imágenes), temporal (como series de tiempo o lenguaje) o cuando se necesita generar información nueva, estas redes no son suficientes. Su estructura fija no aprovecha patrones locales ni dependencias a lo largo del tiempo, y esto limita su rendimiento en muchos escenarios reales.

Por ese motivo, se han desarrollado nuevas arquitecturas diseñadas específicamente para ciertos tipos de datos o tareas. Algunas redes están pensadas para trabajar con secuencias, otras para comprimir o reconstruir información, y otras incluso para generar contenido nuevo como texto, audio o imágenes. Cada arquitectura tiene sus propias características y ventajas, y elegir la adecuada puede marcar una gran diferencia en el rendimiento del modelo.

Límites de las redes feedforward

Las redes feedforward, como las MLP, funcionan muy bien cuando los datos se pueden representar como vectores independientes entre sí, sin necesidad de tener en cuenta el orden o la relación entre elementos. Sin embargo, en muchos problemas reales esta independencia no existe.

Por ejemplo, en tareas de procesamiento de texto, audio o series temporales, el orden en el que aparecen los datos es fundamental. Una MLP trata cada entrada de forma aislada, sin memoria del pasado ni contexto, por lo que pierde información clave en tareas secuenciales. Lo mismo ocurre con imágenes, donde las relaciones espaciales entre píxeles son esenciales para entender el contenido. Una red totalmente conectada trata todos los píxeles como independientes, ignorando su posición relativa.

Otro límite importante es que las MLP no tienen mecanismos internos para detectar patrones repetitivos, dependencias a largo plazo o relaciones jerárquicas. Todo debe aprenderse desde cero a partir de los datos de entrenamiento, lo que las vuelve ineficientes en muchos contextos.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Además, a medida que se intenta aumentar su capacidad añadiendo más capas o neuronas, crece rápidamente el número de parámetros, lo que dificulta el entrenamiento y eleva el riesgo de sobreajuste. Aunque funcionan como una base sólida para muchos modelos, no son la mejor opción cuando los datos tienen estructura compleja.

Por todo esto, han surgido otras arquitecturas que superan estas limitaciones gracias a diseños adaptados a la estructura de los datos y a mecanismos específicos para capturar relaciones más ricas.

Por qué se necesitan arquitecturas adaptadas

Cada tipo de dato tiene una estructura interna distinta. El texto tiene un orden secuencial, las imágenes tienen organización espacial, el audio cambia a lo largo del tiempo, y en muchos casos los datos contienen ruido o están incompletos. Una arquitectura como la MLP, que trata los datos como vectores planos y sin estructura, no es capaz de aprovechar estas particularidades.

Las arquitecturas adaptadas surgen como respuesta a estas necesidades. Se diseñan pensando en cómo fluye la información en el tipo de dato que se quiere tratar. Por ejemplo, las redes convolucionales (CNN) se basan en filtros que recorren la imagen para captar patrones locales y mantener la posición relativa entre píxeles. Las redes recurrentes (RNN) y sus variantes, como LSTM, están pensadas para memorizar el pasado y entender cómo evoluciona una secuencia en el tiempo. Los autoencoders permiten comprimir y reconstruir información, y las GAN pueden generar ejemplos nuevos a partir de ruido.

Estas redes no solo imitan mejor la estructura de los datos, sino que también aprovechan esa estructura para entrenarse de forma más eficiente y con mejor rendimiento. Adaptar la arquitectura al tipo de problema es una forma de introducir conocimiento previo en el modelo, lo que reduce la carga de aprendizaje y mejora los resultados.

Por eso, en lugar de forzar una red MLP a resolver cualquier tarea, se desarrollan arquitecturas especializadas que permiten afrontar con éxito problemas donde las redes tradicionales no son suficientes.

Relación entre tipo de datos y tipo de red

No todas las redes neuronales sirven para todos los problemas. Elegir la arquitectura adecuada depende en gran medida del tipo de datos con los que se trabaja y de la tarea que se quiere resolver. A continuación se resumen algunas relaciones habituales entre tipo de datos y redes más eficaces.

Cuando se trabaja con datos tabulares, es decir, filas y columnas como en una tabla de Excel, donde cada fila es una observación y cada columna una variable, las MLP son una opción muy adecuada. Su estructura es sencilla, pero suficiente para aprender relaciones complejas entre variables numéricas o categóricas.

Para datos con estructura espacial, como imágenes, señales biomédicas o mapas, se usan redes convolucionales (CNN). Estas redes aplican filtros que capturan patrones locales y respetan la organización del espacio. Por eso son especialmente potentes en tareas como el reconocimiento de objetos o la clasificación de imágenes.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Cuando el dato tiene una estructura secuencial o temporal, como ocurre con texto, audio, series temporales o lenguaje natural, se utilizan redes recurrentes (RNN) o sus variantes mejoradas como LSTM o GRU. Estas redes están diseñadas para mantener una "memoria" del pasado y procesar la información en orden, lo que permite captar dependencias a lo largo del tiempo.

Si el objetivo es comprimir la información, reconstruir datos incompletos o detectar anomalías, los autoencoders son una buena opción. Su arquitectura está pensada para aprender una representación interna más compacta que conserve lo más importante del dato original.

Por último, cuando se trata de generar ejemplos nuevos, como imágenes sintéticas, textos o audios inventados, las redes generativas como las GAN permiten crear contenido nuevo a partir de ruido, imitando el estilo o distribución de los datos originales.

Cada tipo de red aprovecha la estructura específica del dato, lo que la hace más eficiente y precisa en su contexto. Por eso es importante conocer las diferencias y saber cuándo conviene utilizar una u otra.

Redes neuronales CNN

Las redes convolucionales son un tipo específico de arquitectura pensada para procesar datos que tienen una estructura espacial, como imágenes. A diferencia de las redes MLP, que tratan cada entrada de forma independiente, las CNN pueden aprovechar las relaciones locales entre píxeles o valores cercanos. Esto les permite detectar patrones visuales como bordes, formas o texturas de forma eficiente.

Qué son y para qué se utilizan

Una red neuronal convolucional está diseñada para procesar datos en forma de cuadrícula, como una imagen compuesta por píxeles. Aprovecha la estructura local del dato para aprender representaciones compactas y efectivas, capturando patrones espaciales que se repiten en distintas partes del conjunto de entrada. Por eso se usan principalmente en tareas de visión por computador como clasificación de imágenes, detección de objetos, reconocimiento facial o diagnóstico médico. También se aplican en señales de audio y series temporales unidimensionales.

Estructura típica de una CNN

Una red convolucional está formada por una secuencia de capas especializadas que transforman progresivamente la imagen de entrada en una representación más abstracta. La estructura más habitual empieza con una o más capas convolucionales, que aplican filtros o kernels para detectar patrones locales como bordes o formas. Después suelen incluirse capas de agrupamiento (pooling), que reducen la resolución manteniendo la información esencial, y finalmente una o más capas densas que permiten tomar decisiones basadas en las representaciones extraídas.

Este tipo de red sigue siendo una arquitectura feedforward, donde la información fluye de entrada a salida sin bucles, pero está optimizada para aprovechar la estructura espacial del dato. Normalmente, cuanto más profunda la red, más complejos son los patrones que puede aprender.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Capas principales y su función

Las CNN se construyen a partir de varios tipos de capas, cada una con una función muy específica:

- La capa convolucional es la más característica. Aplica filtros que se deslizan sobre la imagen para detectar patrones locales. Cada filtro responde a características distintas, como líneas horizontales, esquinas o texturas. A medida que se añaden más capas, los filtros se vuelven más complejos y especializados.
- La capa de activación introduce no linealidad tras la convolución, normalmente usando funciones como ReLU. Esto permite a la red aprender relaciones más complejas y no limitarse a combinaciones lineales de los píxeles.
- La capa de pooling o submuestreo reduce la dimensión de los datos agregando la información en regiones vecinas, lo que disminuye el coste computacional y mejora la robustez del modelo. La más habitual es max pooling, que conserva el valor más alto en cada zona.
- Finalmente, las capas densas conectan todas las neuronas entre sí. Estas capas están al final de la red y transforman las representaciones extraídas en una predicción final, como una etiqueta o un valor numérico.

Comparativa con redes MLP

Las MLP procesan los datos de forma completamente conectada, es decir, cada neurona se conecta con todas las de la siguiente capa. Esto funciona bien con vectores de entrada, pero no aprovecha la estructura espacial de datos como imágenes. Además, requiere muchas conexiones y parámetros, lo que hace que escale mal con entradas grandes.

En cambio, las CNN están diseñadas para trabajar con datos que tienen estructura espacial, como imágenes o vídeos. Utilizan filtros que se aplican localmente, lo que permite detectar patrones visuales como bordes o texturas. Gracias a esta arquitectura, las CNN necesitan menos parámetros y pueden aprender representaciones mucho más eficientes.

Otra diferencia importante es que las CNN combinan extracción de características y clasificación en una sola red, mientras que en modelos basados en MLP suele hacerse por separado. Esta capacidad de aprender automáticamente qué patrones son importantes para la tarea es una de las claves de su éxito en visión artificial.

Aplicaciones más habituales

Las CNN se han convertido en el modelo de referencia para el tratamiento de imágenes y otros datos espaciales. Se utilizan en tareas de clasificación de imágenes, como reconocer objetos, personas o animales. También se aplican al reconocimiento facial, detección de emociones, análisis de rayos X y otras imágenes médicas.

Otras aplicaciones comunes incluyen la detección de objetos dentro de una imagen, segmentación semántica (donde cada píxel se clasifica), generación de imágenes, restauración de imágenes dañadas o eliminación de ruido. Además, se utilizan en vídeo para seguimiento de objetos, reconocimiento de acciones o incluso análisis del tráfico.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



También pueden aplicarse en campos que no son visuales si los datos se representan como imágenes, por ejemplo, espectrogramas en audio o mapas de calor en sensores. Esta versatilidad ha hecho que las CNN estén presentes en gran parte de las soluciones basadas en deep learning actuales.

Ejemplo red neuronal CNN

Este ejemplo muestra cómo construir una red neuronal convolucional (CNN) para una tarea de clasificación de imágenes. Este tipo de red es muy eficaz para datos que tienen estructura espacial, como imágenes o vídeos. A diferencia de una MLP, que trata todos los píxeles como independientes, una CNN puede detectar patrones locales como bordes, formas o texturas, gracias a sus capas especiales.

A continuación, se construye una CNN sencilla para clasificar imágenes de dígitos escritos a mano (como los del conjunto MNIST), incluyendo algunas de las capas más comunes y técnicas para mejorar su rendimiento.

```
model = Sequential([
    Conv2D(32, kernel_size=(3,3), activation='relu', input_shape=(28, 28,
1)),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(64, kernel_size=(3,3), activation='relu'),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

model.compile(optimizer=Adam(), loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_data=(X_test, y_test))
```

- **Conv2D**: estas capas aplican filtros sobre pequeñas regiones de la imagen para detectar patrones locales. En este caso, la primera capa usa 32 filtros y la segunda 64, ambos de tamaño 3x3. La activación **relu** introduce no linealidad.
- **MaxPooling2D**: reduce la dimensión espacial de las imágenes intermedias seleccionando solo el valor máximo de cada región (2x2). Así se consigue reducir el número de parámetros y se conservan las características más importantes.
- **Flatten**: aplanar la salida tridimensional de las capas anteriores en un vector que se puede conectar con las capas densas finales.
- **Dense(128)**: una capa totalmente conectada que transforma la salida anterior en una representación más abstracta. Se usa **relu** para seguir captando relaciones no lineales.
- **Dropout(0.5)**: durante el entrenamiento, desconecta aleatoriamente el 50% de las neuronas de esta capa. Esto reduce el riesgo de sobreajuste.
- **Dense(10, softmax)**: capa de salida con una neurona por clase (dígitos del 0 al 9). **Softmax** convierte las salidas en probabilidades para clasificar la imagen.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Este modelo incluye muchas capas y técnicas a la vez para mostrar cómo se puede estructurar una CNN completa. En problemas reales, puede no ser necesario incluir tantas, y se recomienda empezar con una red más simple.

Redes neuronales recurrentes RNN y LSTM

Hasta ahora hemos trabajado con redes MLP, que procesan los datos de forma independiente, sin tener en cuenta el orden o la relación temporal entre las entradas. Sin embargo, en muchos problemas reales como la predicción de series temporales, el análisis de texto o el reconocimiento de voz, el contexto y la secuencia son fundamentales. Para abordar este tipo de tareas, necesitamos redes neuronales que puedan recordar información previa y utilizarla durante el procesamiento.

Las redes neuronales recurrentes (RNN) fueron desarrolladas precisamente para cubrir esta necesidad. Incorporan mecanismos que les permiten mantener un estado interno, como una forma de memoria, que se actualiza paso a paso. A partir de esta base, surgieron variantes más potentes como las redes LSTM (Long Short-Term Memory), capaces de aprender dependencias a largo plazo de forma estable. En esta sección exploraremos cómo funcionan, sus ventajas, limitaciones y las aplicaciones más comunes.

Cómo procesan secuencias

Las redes neuronales recurrentes son un tipo de red diseñado para trabajar con datos secuenciales. A diferencia de las MLP, que procesan cada entrada de forma independiente, las RNN incorporan conexiones internas que permiten a cada neurona almacenar un pequeño estado de memoria. Ese estado se actualiza en cada paso y se transmite a la siguiente entrada, lo que permite que la red tenga en cuenta el contexto previo. Este mecanismo permite procesar secuencias de cualquier longitud, ya que la red puede reutilizar la misma estructura para cada paso del tiempo.

Este tipo de arquitectura es especialmente útil en tareas donde el orden de los datos importa. Por ejemplo, al analizar texto o señales, la red no solo necesita saber qué dato llega, sino también en qué posición lo hace y qué ha aparecido antes.

Limitaciones de las RNN básicas

Las RNN simples pueden funcionar bien en secuencias cortas, pero presentan dos problemas graves cuando la secuencia es larga. El primero es la desaparición del gradiente, que impide que los pesos se actualicen correctamente al entrenar, y el segundo es la explosión del gradiente, que hace que los valores se descontrolen. Ambos efectos dificultan que las RNN aprendan relaciones a largo plazo entre elementos separados de la secuencia. Por este motivo, aunque las RNN introdujeron la idea de memoria en redes neuronales, han sido superadas en muchos contextos por variantes mejoradas.

LSTM y su capacidad para mantener memoria

Las redes LSTM (Long Short-Term Memory) se diseñaron precisamente para solucionar los límites de las RNN tradicionales. Incorporan un sistema de compuertas que controla de forma explícita qué información se guarda, qué se olvida y qué se transmite. Gracias a ello, son capaces de mantener

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



memoria durante más pasos de la secuencia y aprender dependencias a largo plazo de forma estable. Cada célula LSTM contiene tres compuertas: una de entrada, una de olvido y una de salida. Estas compuertas regulan el flujo de datos y permiten que la red conserve solo lo relevante y descarte lo que no aporta información.

Aplicaciones habituales

Las redes RNN y LSTM se utilizan en muchas tareas con datos secuenciales. Algunas de las más frecuentes son el análisis de series temporales (por ejemplo, predicción de precios), el procesamiento del lenguaje natural (traducción automática, análisis de sentimientos, generación de texto), y la clasificación de secuencias en audio o vídeo. Keras ofrece capas específicas como `SimpleRNN`, `LSTM` y `GRU`, que permiten construir este tipo de modelos fácilmente y adaptarlos al tipo de secuencia y problema.

Estructura básica de las redes recurrentes y diferencias con MLP

Las redes neuronales recurrentes (RNN) y sus variantes como LSTM comparten ciertos elementos estructurales con las redes multicapa MLP, pero también introducen diferencias clave. En una red recurrente, la entrada no se procesa toda de golpe como ocurre en las redes feedforward, sino que se analiza paso a paso a lo largo de una secuencia. Esto permite que la red mantenga una especie de memoria temporal, gracias a conexiones internas que transmiten información de un paso al siguiente.

El esquema básico suele comenzar con una capa recurrente como `SimpleRNN` o `LSTM`, que se encarga de capturar relaciones entre elementos consecutivos. Esta capa puede ir seguida de una o más capas densas (`Dense`) que refinan la representación antes de generar la salida. Este tipo de estructura es común en tareas como predicción de series temporales o procesamiento de texto.

RNN y MLP comparten varios hiperparámetros. Por ejemplo, el número de épocas, el tamaño del batch y la tasa de aprendizaje son comunes y afectan tanto al tiempo de entrenamiento como al rendimiento. También es posible usar técnicas de regularización como Dropout o L2, elegir funciones de activación o configurar distintos optimizadores.

Pero las redes recurrentes introducen hiperparámetros específicos que controlan su comportamiento secuencial. Entre ellos, el número de unidades recurrentes determina la capacidad de memoria, el tamaño de los pasos de la secuencia (`timesteps`) indica cuántos elementos se procesan en cada ejemplo, y el parámetro `return_sequences` permite decidir si se devuelve una salida por cada paso o solo una al final. Las redes LSTM, además, incorporan puertas que aprenden a mantener o olvidar información de forma controlada, lo que evita que el gradiente desaparezca durante el entrenamiento con secuencias largas.

Las RNN simples pueden ser útiles en secuencias cortas o con patrones simples, pero cuando existen dependencias a largo plazo, las redes LSTM ofrecen mejores resultados. Por eso, en la mayoría de casos prácticos, se prefiere usar LSTM directamente.

Ejemplo red neuronal LSTM

Este modelo está diseñado para resolver un problema de regresión basado en secuencias. Por ejemplo, predecir el precio del siguiente día a partir de los valores anteriores de precio y tiempo.

```
modelo = Sequential([
    LSTM(64, input_shape=(None, 2), kernel_regularizer=l2(0.001),
    recurrent_dropout=0.2, return_sequences=False),
    Dropout(0.3),
    Dense(32, activation='relu', kernel_regularizer=l2(0.001)),
    Dense(1, activation='linear')
])

modelo.compile(optimizer=Adam(learning_rate=0.001),
loss='mean_squared_error', metrics=['mae'])

modelo.fit(X_train_seq, y_train, epochs=20, batch_size=32,
validation_data=(X_val_seq, y_val))
```

La red es una arquitectura LSTM, especialmente útil cuando el orden de las observaciones importa. Aquí se asume que las entradas (`X_train_seq`) tienen forma (n_observaciones, longitud_de_secuencia, 2), donde los dos valores pueden ser, por ejemplo, el precio y la marca temporal.

Primero se usa una capa `LSTM` con 64 unidades. Esta capa tiene regularización L2 y `recurrent_dropout`, lo que ayuda a controlar el sobreajuste incluso dentro de la memoria de la LSTM. A continuación se añade una capa `Dropout` que también elimina conexiones de forma aleatoria durante el entrenamiento.

Después se añade una capa densa de 32 neuronas con activación ReLU y regularización L2. Finalmente, la salida es una única neurona con activación `linear`, lo habitual en problemas de predicción de un valor continuo.

Se compila con el optimizador Adam, la pérdida `mean_squared_error` (adecuada para regresión) y como métrica se calcula el error absoluto medio (`mae`). El modelo se entrena durante 20 épocas con un tamaño de batch de 32 y se valida con un conjunto separado.

Se han incluido varias técnicas como regularización y dropout para que el alumnado vea cómo aplicarlas, aunque no necesariamente todas sean adecuadas a la vez en un caso concreto.

Autoencoders y sus variantes

Los autoencoders son redes neuronales diseñadas para aprender representaciones compactas y eficientes de los datos. A diferencia de otros modelos que predicen una salida distinta a la entrada, los autoencoders aprenden a copiar su propia entrada. Este proceso obliga a la red a descubrir patrones internos y a eliminar la información menos relevante.

Se utilizan principalmente para reducir la dimensión, comprimir, detectar anomalías o limpiar datos. Esta limpieza se refiere a la capacidad de eliminar el ruido, es decir, pequeñas alteraciones o errores

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



que aparecen en los datos originales. Por ejemplo, en imágenes, el ruido pueden ser píxeles distorsionados o detalles sin valor; en datos numéricos, errores de medición o fluctuaciones aleatorias. Al reconstruir la entrada, los autoencoders aprenden a recuperar una versión más limpia, conservando solo lo esencial. Por eso también se utilizan como paso previo en otros modelos, aprovechando la representación comprimida como entrada para tareas más complejas.

Qué es un autoencoder y cómo está estructurado

Un autoencoder es una red neuronal que no intenta predecir una salida diferente, sino reconstruir la misma entrada. Esto le obliga a aprender una versión comprimida pero significativa de los datos.

Su estructura se divide en dos bloques. El encoder transforma la entrada original en una codificación más pequeña, conocida como representación latente. El decoder toma esa codificación y reconstruye la entrada a partir de ella. Ambos bloques pueden estar formados por capas densas o convolucionales, dependiendo del tipo de dato.

En la práctica, el encoder actúa como un compresor y el decoder como un descompresor. Por ejemplo, si una imagen tiene 784 píxeles, el encoder podría reducirla a un vector de 32 valores. El decoder intenta reconstruir los 784 píxeles desde esos 32. El modelo se entrena minimizando la diferencia entre entrada y salida.

Reconstrucción y función de pérdida

El objetivo de un autoencoder es reconstruir su propia entrada. Para saber si lo hace bien, se mide la diferencia entre la entrada original y la salida generada. Esta diferencia se llama función de pérdida, y cuanto menor sea, mejor ha aprendido el modelo.

Lo más habitual es usar el error cuadrático medio para calcular esa pérdida. Si los datos están en formato binario, también puede utilizarse la entropía cruzada. Durante el entrenamiento, el modelo ajusta sus parámetros para minimizar esta pérdida y mejorar su capacidad de reconstrucción.

Aplicaciones típicas como compresión, limpieza y detección

Los autoencoders se usan en muchos contextos prácticos:

- **Compresión de datos:** el encoder reduce la dimensión, y así se puede almacenar menos información manteniendo lo esencial.
- **Eliminación de ruido:** se entrena al modelo con ejemplos con ruido como entrada y datos limpios como salida, para que aprenda a recuperar la información útil.
- **Detección de anomalías:** si el modelo ha aprendido a reconstruir datos normales, fallará con datos extraños y el error de reconstrucción será mucho mayor. Esto permite identificar comportamientos atípicos.

Variantes más habituales

Autoencoders variacionales VAE y su capacidad para generar datos

Los VAE no aprenden una codificación concreta, sino una distribución de probabilidad. Esto les permite generar datos nuevos parecidos a los originales. Por ejemplo, pueden crear imágenes de caras distintas a las vistas durante el entrenamiento. Se usan en generación de imágenes, audio o texto, y permiten manipular las características latentes para explorar el espacio de datos.

Autoencoders convolucionales y su uso con datos espaciales

Estos autoencoders usan capas convolucionales en lugar de densas, lo que les permite trabajar mejor con imágenes, vídeos u otros datos espaciales. Los filtros capturan patrones como bordes o texturas y mantienen la estructura del dato original. Son ideales para tareas de compresión de imágenes, eliminación de ruido visual o generación de contenido visual.

Diferencias clave

Ambos pueden trabajar con imágenes, pero no con el mismo objetivo. Los autoencoders convolucionales buscan reconstruir fielmente la imagen, capturando sus detalles. En cambio, los VAE priorizan crear representaciones estructuradas que permitan generar imágenes nuevas, similares pero no idénticas. Para limpiar o comprimir, los convolucionales son más adecuados. Para crear contenido nuevo, los VAE son la mejor opción.

Ejemplo autoencoder

Aquí tienes un ejemplo de autoencoder diseñado para eliminar ruido en imágenes, utilizando Keras. Se trabaja con imágenes de 28x28 píxeles (como MNIST) que se han distorsionado añadiendo ruido aleatorio. El modelo intenta aprender la versión limpia a partir de la imagen ruidosa. Se utiliza una arquitectura simétrica con capas convolucionales, ideal para capturar la estructura espacial de las imágenes:

```
model = Sequential([Conv2D(32, (3, 3), activation='relu', padding='same',
input_shape=(28, 28, 1)),
                    MaxPooling2D((2, 2), padding='same'),
                    Conv2D(16, (3, 3), activation='relu', padding='same'),
                    MaxPooling2D((2, 2), padding='same'),
                    Conv2D(8, (3, 3), activation='relu', padding='same'),
                    UpSampling2D((2, 2)),
                    Conv2D(16, (3, 3), activation='relu', padding='same'),
                    UpSampling2D((2, 2)),
                    Conv2D(1, (3, 3), activation='sigmoid',
padding='same')]))

model.compile(optimizer='adam', loss='binary_crossentropy')

model.fit(x_train_noisy, x_train, epochs=20, batch_size=128,
validation_split=0.2)
```

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Este autoencoder convolucional está orientado a la eliminación de ruido en imágenes. La entrada (`x_train_noisy`) son imágenes alteradas con ruido gaussiano o aleatorio, y la salida esperada (`x_train`) son las imágenes originales limpias.

Aunque en el código no se define de forma explícita, el modelo contiene dos partes bien diferenciadas. Las primeras capas forman el **encoder**, que extrae las características principales de la imagen y reduce progresivamente su resolución. Esta parte incluye varias capas `Conv2D` seguidas de `MaxPooling2D`, y actúa como compresor de la información relevante.

A continuación, las capas `UpSampling2D` y `Conv2D` que amplían de nuevo la resolución constituyen el **decoder**, encargado de reconstruir la imagen original a partir de la representación comprimida. Juntas, estas dos mitades crean una arquitectura simétrica en forma de embudo.

La última capa utiliza activación `sigmoid` para producir valores entre 0 y 1, adecuado si las imágenes están normalizadas. Esta arquitectura se conoce como **denoising autoencoder**, y permite limpiar imágenes sin que el modelo necesite saber cómo es el ruido. El entrenamiento se basa en minimizar la diferencia entre la imagen reconstruida y la imagen original.

Un autoencoder convolucional y una red neuronal convolucional comparten muchas capas como `conv2D`, `maxPooling2D` o `upSampling2D`, pero tienen funciones distintas. Una red convolucional clásica se utiliza para clasificar imágenes o detectar objetos, por lo que acaba con una capa densa que genera una etiqueta o una predicción. En cambio, un autoencoder convolucional intenta reconstruir la propia imagen de entrada, sin asignarle una clase. Aunque visualmente los modelos puedan parecerse, la diferencia está en el objetivo del aprendizaje. Si la red aprende a clasificar, se trata de una CNN. Si aprende a reproducir su entrada lo mejor posible, es un autoencoder.

Redes generativas adversarias GAN

Las GAN (Generative Adversarial Networks) son un tipo especial de red neuronal que no busca clasificar ni predecir, sino generar datos nuevos que se parezcan a los reales. Se basan en un enfoque creativo y competitivo, en el que dos redes se enfrentan entre sí para mejorar progresivamente. Gracias a esta dinámica, las GAN son capaces de crear imágenes, sonidos o textos que parecen auténticos, aunque no existan en el conjunto original. Se han convertido en una de las herramientas más potentes para tareas de generación de contenido, especialmente cuando se dispone de pocos ejemplos o se necesita aumentar la variedad de datos.

Qué son y para qué se utilizan

Las GAN (Generative Adversarial Networks) son un tipo de red neuronal diseñada para generar datos nuevos y realistas a partir de un conjunto de ejemplos. Su funcionamiento se basa en el entrenamiento simultáneo de dos redes neuronales con objetivos opuestos. Por un lado, el generador intenta crear datos falsos que imiten a los reales, mientras que el discriminador intenta distinguir si una muestra proviene del conjunto real o ha sido generada artificialmente. Este proceso adversarial obliga al generador a mejorar constantemente, hasta producir datos tan convincentes que el discriminador ya no pueda diferenciarlos con claridad.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Las GAN no se utilizan para tareas clásicas como clasificación o regresión, sino para aprender la distribución de los datos y generar nuevos ejemplos a partir de ella. Por eso se aplican en contextos donde se busca crear imágenes sintéticas, aumentar datos (data augmentation), mejorar la resolución de imágenes, restaurar zonas dañadas o generar estilos visuales distintos. También se han empleado en la creación de vídeos, música y textos.

Su capacidad para aprender estructuras complejas sin supervisión directa las convierte en una herramienta muy potente, pero también presentan retos importantes en el entrenamiento, como la inestabilidad o el riesgo de que el generador se limite a producir variaciones mínimas de los mismos ejemplos.

Arquitectura y cómo funciona

Una red GAN se compone de dos redes neuronales que aprenden juntas en un proceso competitivo. La primera es el generador, cuya tarea es crear ejemplos que imiten los datos reales. La segunda es el discriminador, que intenta distinguir entre ejemplos reales y falsos. Ambos modelos se entrenan de forma simultánea pero con objetivos opuestos.

El generador parte de un vector aleatorio y produce un dato sintético, como una imagen o una secuencia de texto. Este resultado se envía al discriminador junto con datos reales del conjunto de entrenamiento. El discriminador intenta adivinar si cada entrada que recibe es real o generada. A medida que el generador mejora, engaña cada vez más al discriminador. A su vez, el discriminador se entrena para ser más preciso.

Este entrenamiento conjunto continúa hasta que el generador produce resultados tan realistas que el discriminador ya no puede diferenciarlos fácilmente. Entonces se dice que la GAN ha alcanzado un equilibrio. Aunque inicialmente se aplicaron sobre todo a imágenes, esta arquitectura también se ha adaptado a otros tipos de datos como texto, audio o vídeo, ajustando las redes internas para procesar ese tipo de información.

Capas y estructura habitual

En una GAN, tanto el generador como el discriminador tienen arquitecturas de red diferentes, adaptadas a sus objetivos. Aunque existen muchas variantes, la estructura típica sigue un patrón bastante común.

El generador suele comenzar con un vector aleatorio de entrada, normalmente tomado de una distribución normal. Este vector pasa por varias capas densas o convolucionales transpuestas (si se trabaja con imágenes), o bien por capas recurrentes o embebidas (si se trata de texto). El objetivo del generador es transformar ese ruido en un dato con la misma forma y características que los reales. Sus capas finales suelen usar activaciones como tanh o sigmoid, según el tipo de dato de salida.

El discriminador recibe como entrada un dato real o generado. Su estructura recuerda a un clasificador: capas densas o convolucionales que reducen progresivamente la dimensión y extraen características del dato. Termina con una única neurona que produce un valor entre 0 y 1, indicando si cree que el dato es real (valor cercano a 1) o falso (valor cercano a 0). Aquí, la activación final suele ser una sigmoid.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Ambas redes se entrenan juntas pero con funciones de pérdida opuestas. El generador quiere maximizar la confusión del discriminador, mientras que el discriminador quiere minimizar su error al distinguir datos reales de falsos.

Aplicaciones habituales

Las GAN se han convertido en una herramienta versátil para generar, transformar o mejorar datos de forma realista. Algunas de las aplicaciones más habituales son:

Generación de imágenes. Es el uso más conocido, con redes capaces de crear rostros sintéticos, obras de arte, ropa o paisajes que no existen, pero que parecen reales

Transferencia de estilos. Permite modificar imágenes aplicando estilos artísticos, por ejemplo convertir una foto real en una pintura al estilo Van Gogh, o transformar un dibujo en una imagen fotográfica

Mejora de resolución. Las GAN pueden generar versiones en alta definición de imágenes de baja calidad, completando detalles que faltan

Creación de datos sintéticos. En campos donde hay pocos datos disponibles, las GAN se utilizan para generar ejemplos artificiales que ayudan a entrenar otros modelos, como imágenes médicas, textos o muestras de voz

Generación de texto. Algunas variantes de GAN adaptadas a datos secuenciales pueden generar frases, titulares o diálogos similares a los reales, aunque este campo ha sido superado en muchos casos por transformadores como GPT

Detección de anomalías. También se usan como herramienta indirecta, comparando datos reales con los generados para detectar desviaciones y anomalías, útil en seguridad o mantenimiento predictivo

Ejemplo básico de red GAN en Keras

Este ejemplo simplificado genera imágenes parecidas a dígitos del dataset MNIST. Se define un generador que crea imágenes a partir de ruido, y un discriminador que intenta distinguirlas de las reales. El entrenamiento alterna ambas redes:

```
# Generador
generador = Sequential([Dense(128, activation='relu', input_dim=100),
                        BatchNormalization(),
                        Dense(784, activation='sigmoid'),
                        Reshape((28, 28, 1))])

# Discriminador
discriminador = Sequential([Flatten(input_shape=(28, 28, 1)),
                            Dense(128, activation='relu'),
                            Dropout(0.3),
                            Dense(1, activation='sigmoid')])

# Compilar discriminador
discriminador.compile(optimizer='adam', loss='binary_crossentropy')
```

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



```
# Red adversaria
discriminador.trainable = False
gan_input = Input(shape=(100,))
gan_output = discriminador(generator(gan_input))
gan = Model(gan_input, gan_output)
gan.compile(optimizer='adam', loss='binary_crossentropy')
```

Este código define las dos partes de una GAN. El generador transforma un vector de ruido en una imagen. El discriminador intenta distinguir entre imágenes reales y generadas. Al combinar ambos modelos, el generador aprende a engañar al discriminador. Aunque el código real necesita un bucle de entrenamiento propio, este fragmento permite visualizar cómo se estructura una GAN.

Comparativa entre arquitecturas

Arquitectura	Tipos de Datos	Arquitectura básica	Usos habituales
MLP	Datos tabulares	Capas densas conectadas	Clasificación, regresión, predicción con datos simples
RNN	Datos secuenciales	Capas recurrentes	Análisis de texto, series temporales, voz
LSTM	Secuencias largas	Capas recurrentes con memoria	Predicción de tendencias, generación de texto
CNN	Imágenes, señales	Capas convolucionales y pooling	Visión artificial, reconocimiento de patrones visuales
Autoencoder	Imágenes, vectores	Encoder y decoder simétricos	Compresión, limpieza de datos, detección de anomalías
VAE	Imágenes, vectores	Encoder y decoder con ruido	Generación de imágenes, representación latente
GAN	Imágenes, texto, voz	Generador y discriminador	Creación de contenido nuevo, mejora de calidad

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)

